DSA 8020 R Lab 10: Interpolation of Spatial Data

Saransh Rakshak

Due: Thurs. April 24

Contents


```
library(ggplot2)
library(gstat)
library(sp)
```

---

1. Load the data into R. How many observations are in this spatial data set?

Code:

```
load("data/RLab10.RData")
dim(s)
```

```
## [1] 200   2
```

Answer:

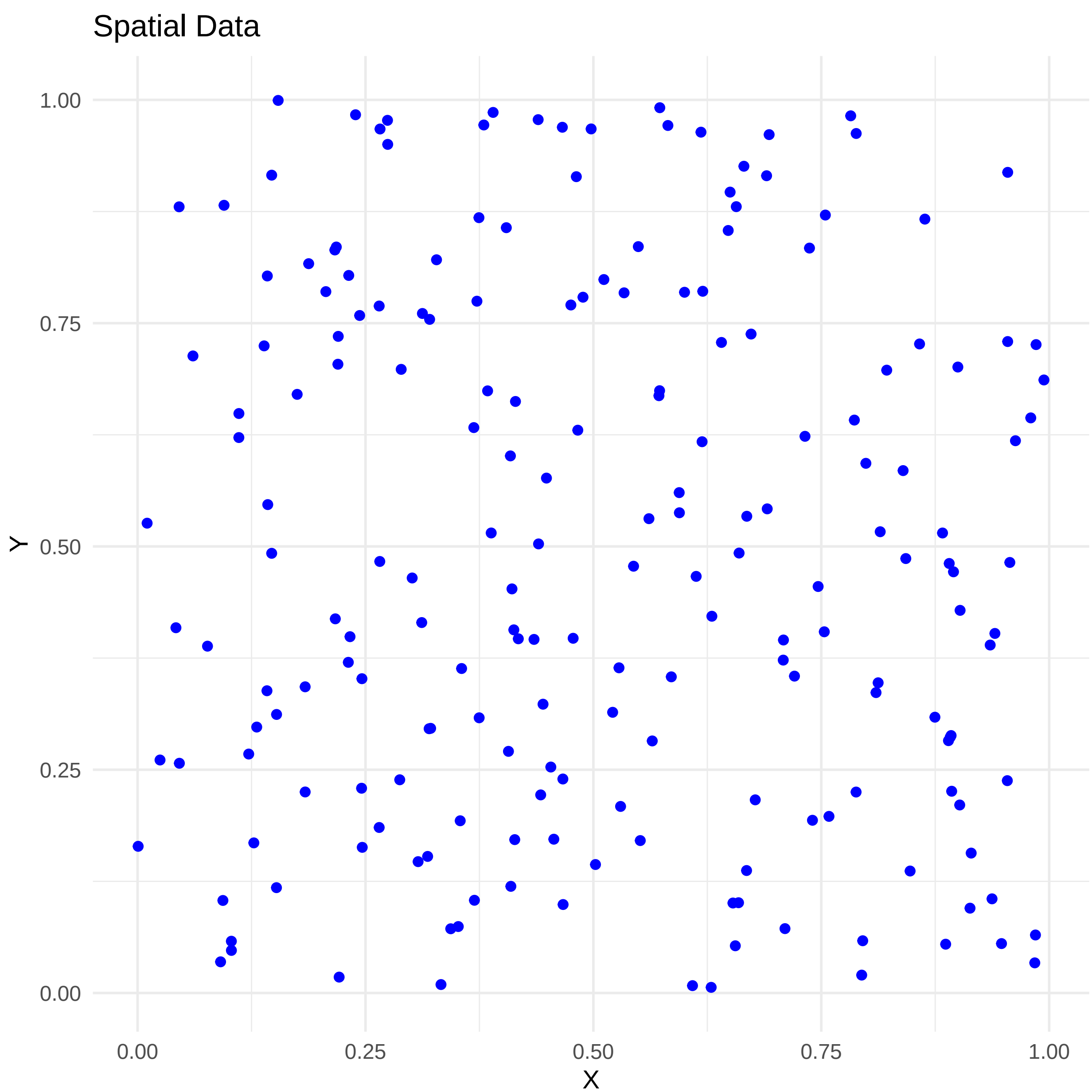
There are 200 observations in spatial data set 's'.

---

2. Plot the spatial data.

Code:

```r
ggplot(s, aes(x= x, y= y)) +
  geom_point(color = "blue") +
  labs(title= "Spatial Data",
       x= "X",
       y= "Y"
       ) +
  theme_minimal()
```

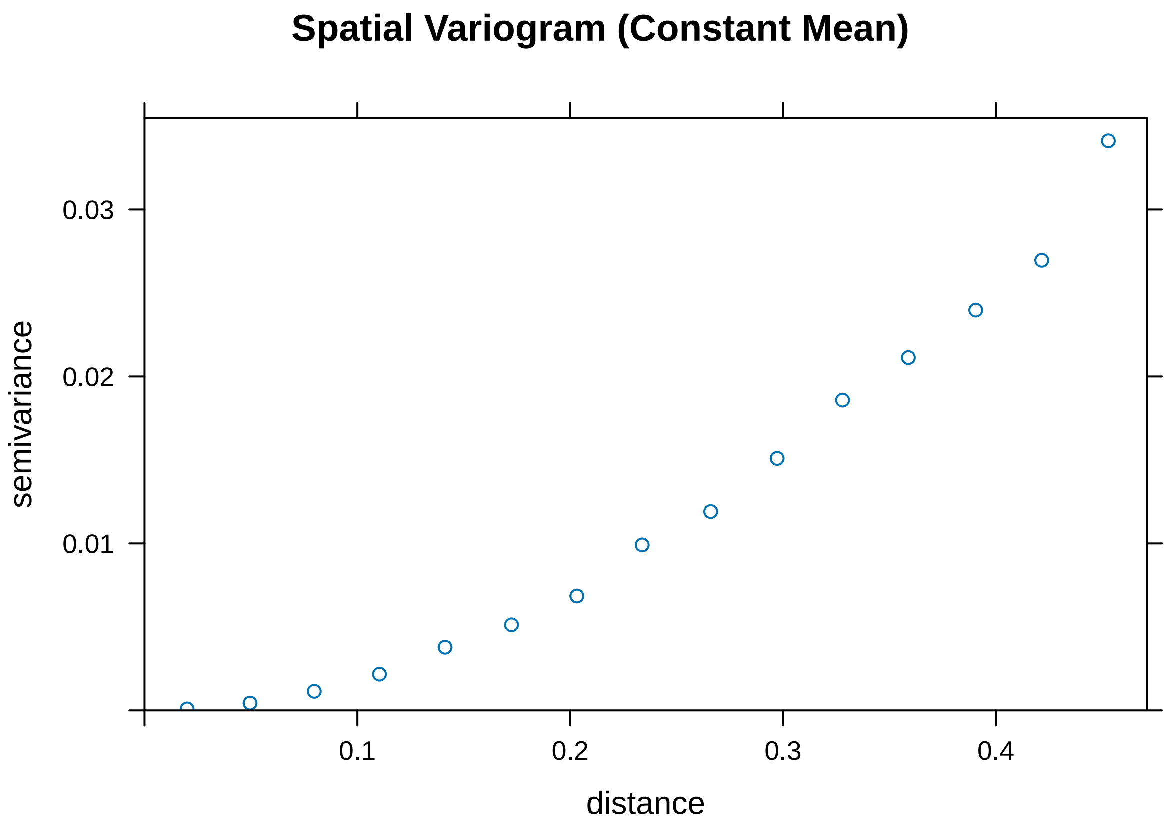


---

3. Plot the spatial variogram and briefly describe your findings.

Code:

```
df <- as.data.frame(s)
colnames(df) <- c("x", "y")
df$z <- sin(df$x + df$y)
coordinates(df) <- ~ x + y

# variogram, mean func constant mean=1
v.emp <- variogram(z ~ 1, data= df)
plot(v.emp, main= "Spatial Variogram (Constant Mean)")
```



Answer:

- Shows increasing semivarience as distance increases, showing positive spatial autocorrelation.
- Little to no nuggeting, shows low noise for measurements.
- No still of curve, shows data does not yet have a spatial dependence. Will need to extend distances to greater than currently sampled to see signs of spatial dependence.

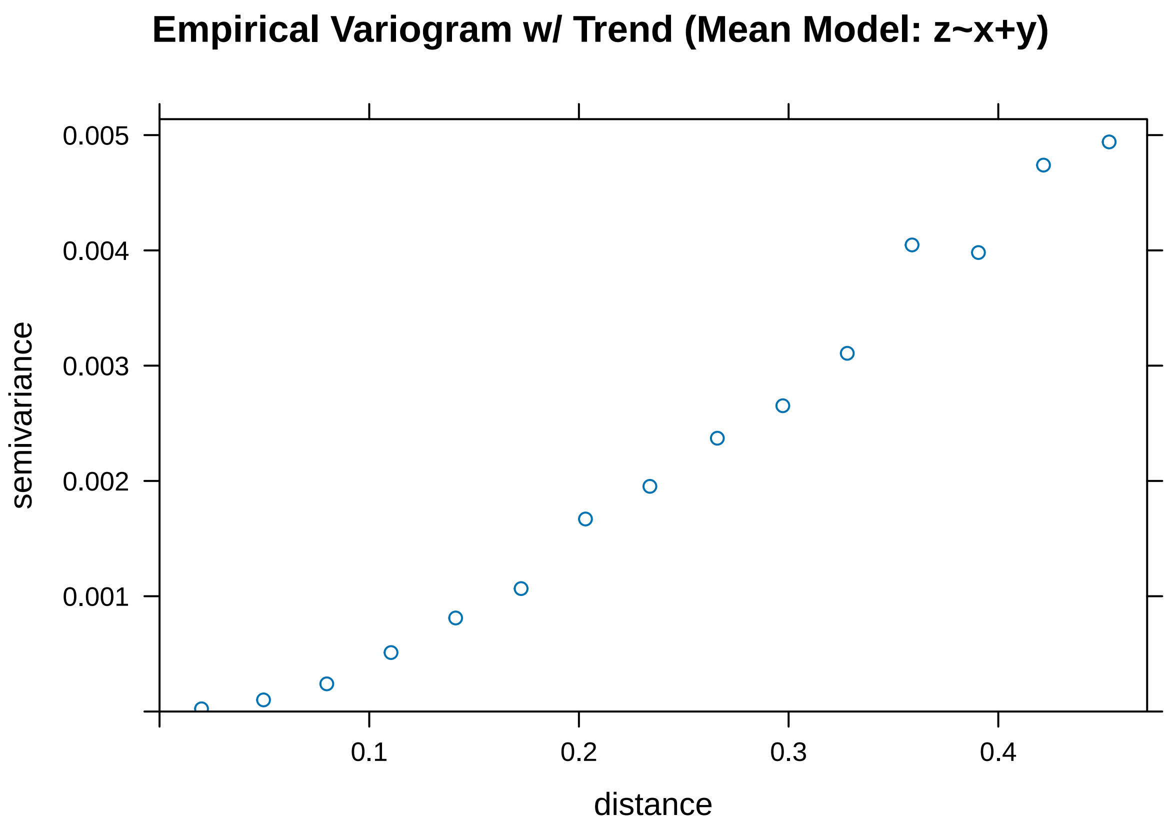
---

4. Fit a Spatial Gaussian process model to the data. Describe the models for the mean function and the covariance function, and report the estimated parameters.

Code:

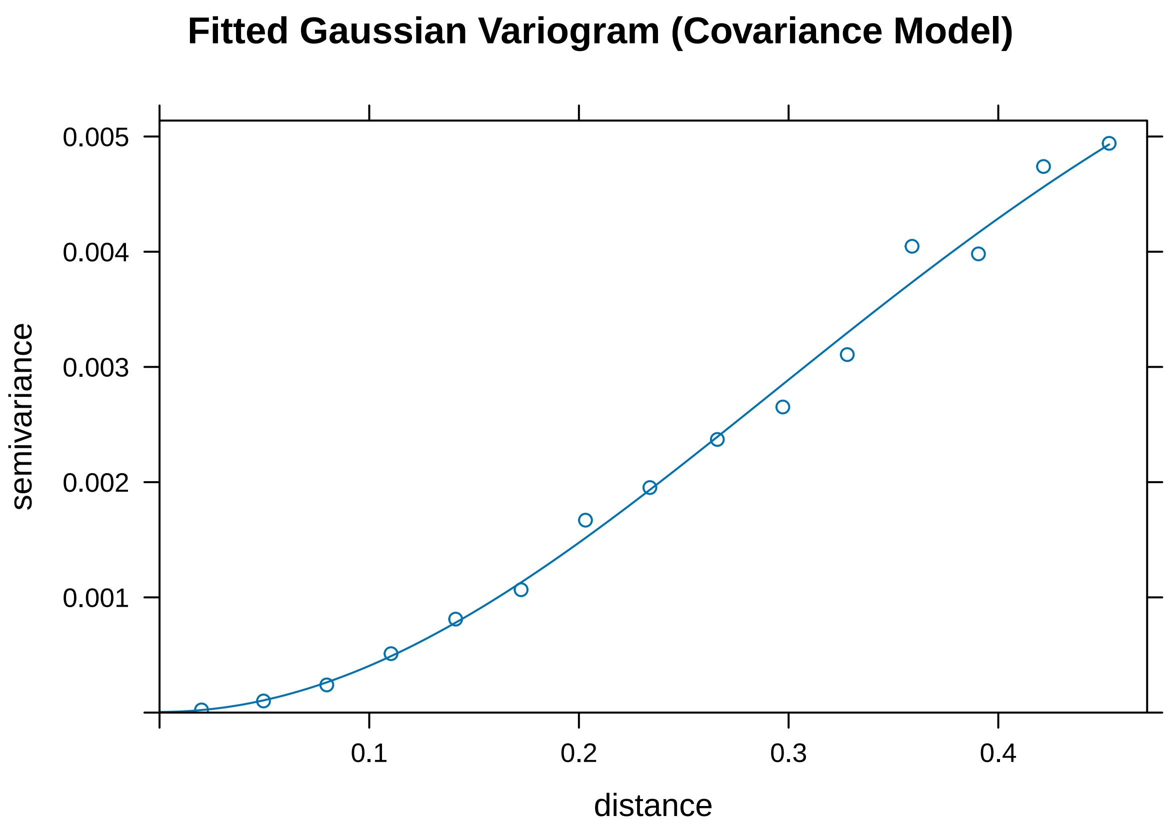
Mean Function Model

```r
# mean function model - linear trend surface model: z~x+y
v.emp <- variogram(z ~ x + y, data = df)
plot(v.emp, main = "Empirical Variogram w/ Trend (Mean Model: z~x+y)")
```



Covariance Function Model

```r
# fit Gaus to empirical variogram
v.fit <- fit.variogram(v.emp, model = vgm(psill = 0.03,
                                          model = "Gau",
                                          range = 0.2,
                                          nugget = 0) )

plot(v.emp, model = v.fit, main = "Fitted Gaussian Variogram (Covariance Model)")
```




```r
# estimated covariance vals
print(v.fit)
```

```
##   model        psill     range
## 1   Nug 5.482179e-06 0.0000000
## 2   Gau 7.059960e-03 0.4140399
```

Answer:

Mean Function Model

Model: $\mu(s) = \beta_0 + \beta_1 x + \beta_2 y$

Code: variogram(z ~ x + y, data = df)

- $\mu(s)$ is our expected mean value of spatial process at distance (s).
- β_0 baseline when x and y equal 0.
- β_1 effect of x on mean.
- β_2 effect of y on mean.

Covariance Function Model

Model: $Cov(h) = \sigma^2 exp(-(h/\theta)^2)$

Code: fit.variogram(v.emp, model = vgm(psill = 0.03, model = "Gau", range = 0.2, nugget = 0))

- σ_2: partial sill.
- θ: range.
- nugget: variance at distance 0.

Estimated Parameters

- Nugget: 5.482179e-06 very small, negligible.
- Partial Sill (σ^2): 7.059960e-03, is our variance of the residual spatial process after accounting for the z~x+y large scale trend. Value is extremely small, expected since trend surface already explains most of variation.
- Range (θ): 0.4140399, shows points smaller than 0.414 distance units are strongly correlated. Points with greater distance units apart have rapidly declining correlation.

Overall Conclusion

Smooth decay in Gaussian model shows strong local spatial continuity. Our trend model $z \sim z + y$ captured most of spatial variation, explains why partial sill and nugget have such small values. Gaussian covariance function continues to explain local correlation in residuals (within a 0.414 distance apart). Overall shows good specification in spatial model - our trend explains the big picture while covariance shows local detail.

---

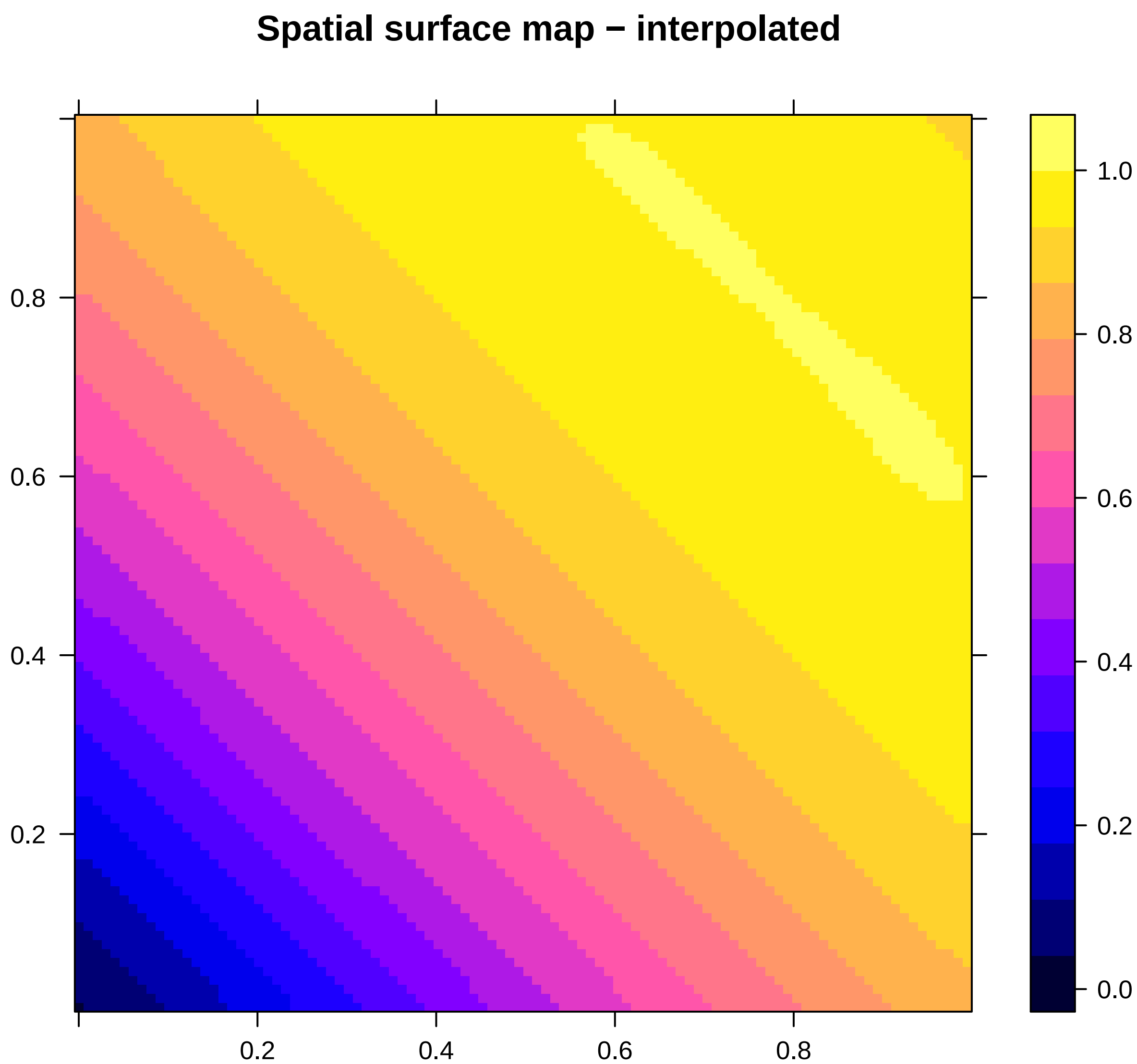
5. Produce the spatial interpolated map with its associated estimation uncertainty.

Code:

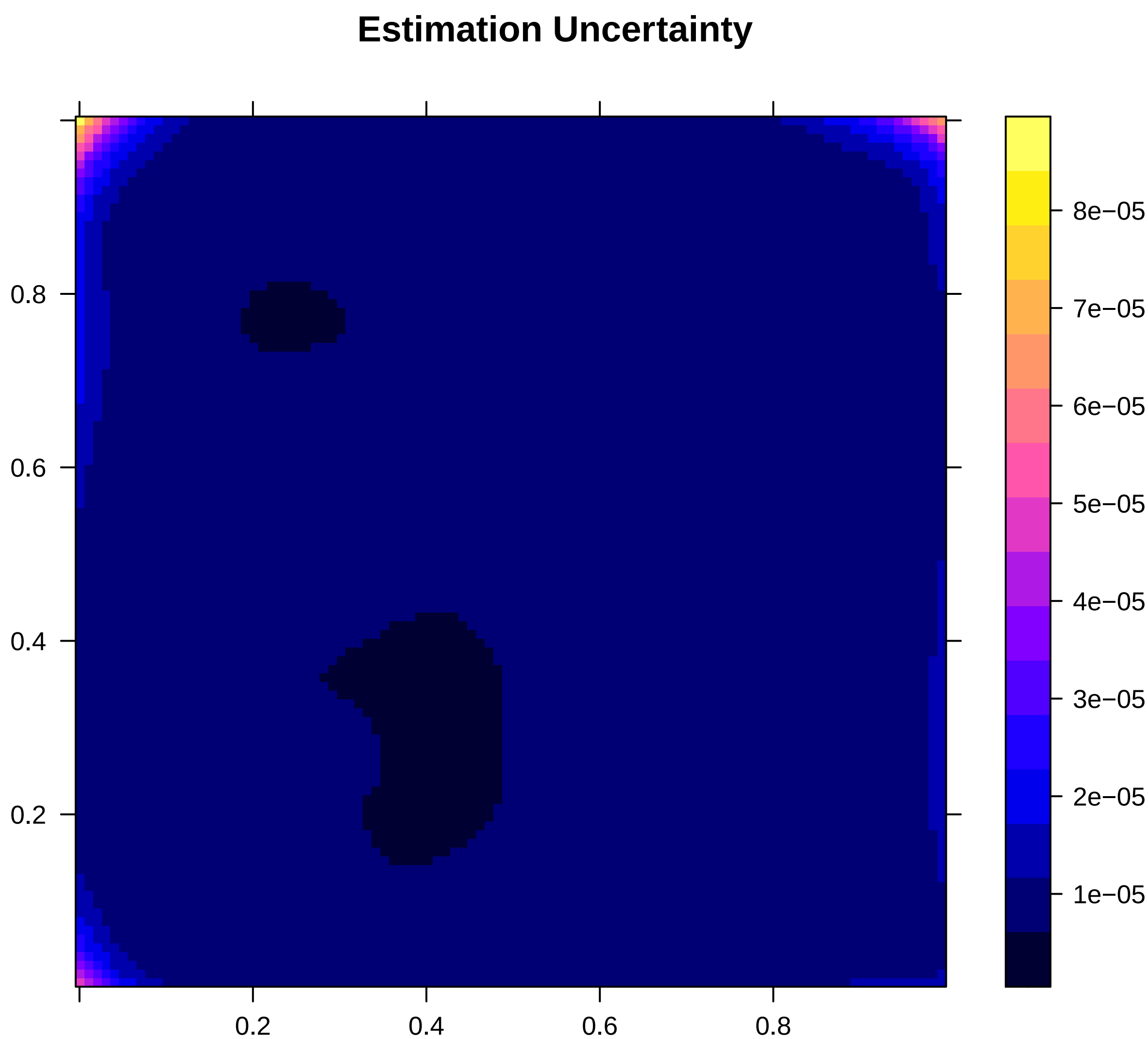
```
# grid to interpolate over
x.range <- range(df@coords[,1])
y.range <- range(df@coords[,2])

v.grd <- expand.grid(
  x = seq(x.range[1], x.range[2], length.out = 100),
  y = seq(y.range[1], y.range[2], length.out = 100)
)

coordinates(v.grd) <- ~x+y
gridded(v.grd) <- TRUE  # spatial grid

# gstat obj - kriging upred
v.g <- gstat(id = "z", formula = z ~ 1, data = df, model = v.fit)
v.kriged <- predict(v.g, newdata = v.grd)
```

```
## [using ordinary kriging]
```

```r
# interpolated
spplot(
  v.kriged["z.pred"],
  main = "Spatial surface map - interpolated",
  scales = list(draw = TRUE)
  )
```




```r
# uncertainty (var) estimate
spplot(
  v.kriged["z.var"],
  main = "Estimation Uncertainty",
  scales = list(draw = TRUE)
  )
```



Estimation Uncertainty
0.8
0.6
0.4
0.2
0.2
0.4
0.6
0.8
8e−05
7e−05
6e−05
5e−05
4e−05
3e−05
2e−05
1e−05